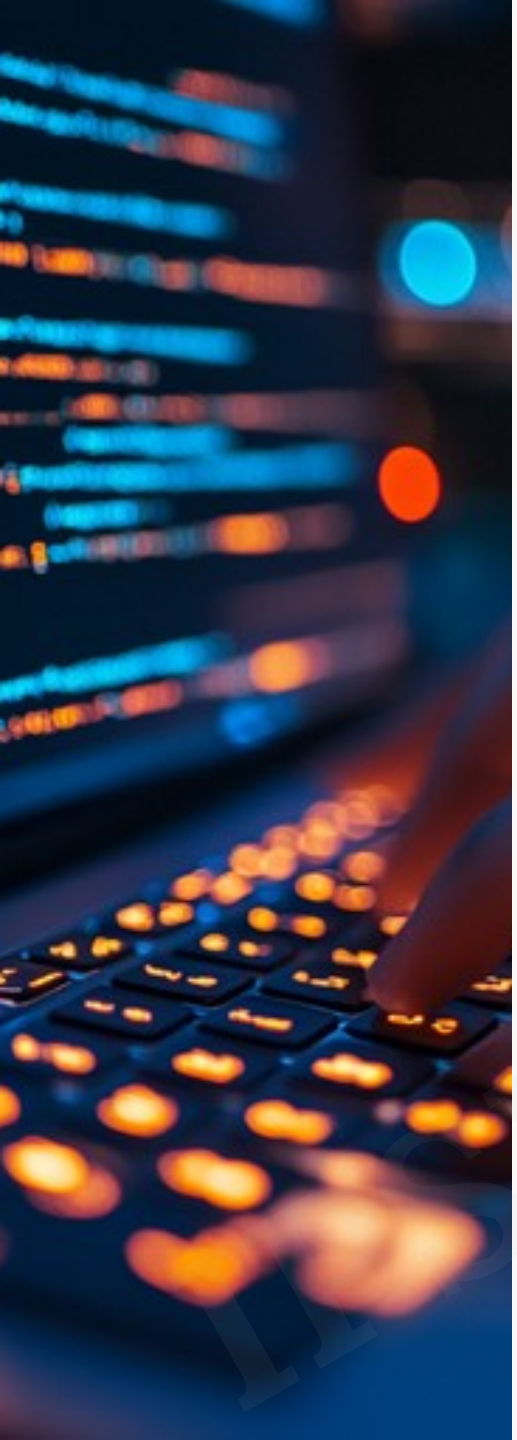


第 2 章、電腦的硬體結構

李建緯
雲科大資工系 智慧感知與系統整合實驗室
IPSI Lab, Yuntech-CSIE

IPSI Lab, Yuntech-CSIE

CPU0 的運作原理

載入與儲存指令

IPSI Lab, Yuntech-CSIE

載入與儲存指令

基本概念：

- **位元組 (Byte)**：計算機中的基本存儲單位，1 Byte = 8 位元 (bits)。
- **字組 (Word)**：在 32 位元計算機中，1 字組 = 32 位元 (4 Byte)。
- **32 位元處理器** 的內部匯流排是 32 位元的，資料的傳輸單位也是 32 位元 (4 Byte) 為一個字組。

載入儲存指令的重要性

- **CPU0** 是精簡指令集 (RISC) 的 CPU，僅有少數指令可直接讀取或寫入記憶體中的數值。這些指令稱為 **記憶體存取指令**，是計算機操作的核心。

IPSI Lab, Yuntech-CSIE

記憶體存取指令集

CPU0 的指令集中包含以下 8 個記憶體存取指令：

1. **LD (Load Word)**：載入字組
2. **ST (Store Word)**：儲存字組
3. **LDB (Load Double Byte)**：載入雙位元組
4. **STB (Store Byte Register)**：儲存位元組
5. **LDR (Load Register)**：載入暫存器
6. **STR (Store Register)**：儲存暫存器
7. **LBR (Load Byte Register)**：載入位元組
8. **SBR (Store Byte Register)**：儲存位元組暫存器

IPSI Lab, Yuntech-CSIE

LD 和 LDB 指令 (載入指令)

- **LD (Load) 指令**
  - 將一個完整的字組 (32位元)載入到暫存器中。
  - 格式: `LD Rd, [address]`
    - **Rd**：目標暫存器，用來儲存載入的數值。
    - **address**：記憶體中的地址，從該地址載入數值。
- **LDB (Load Byte) 指令**
  - 作用：載入一個位元組 (8位元) 到暫存器中。
  - 用途：適合需要精細操作的小資料存取，如單一字元或標記位元組。

IPSI Lab, Yuntech-CSIE

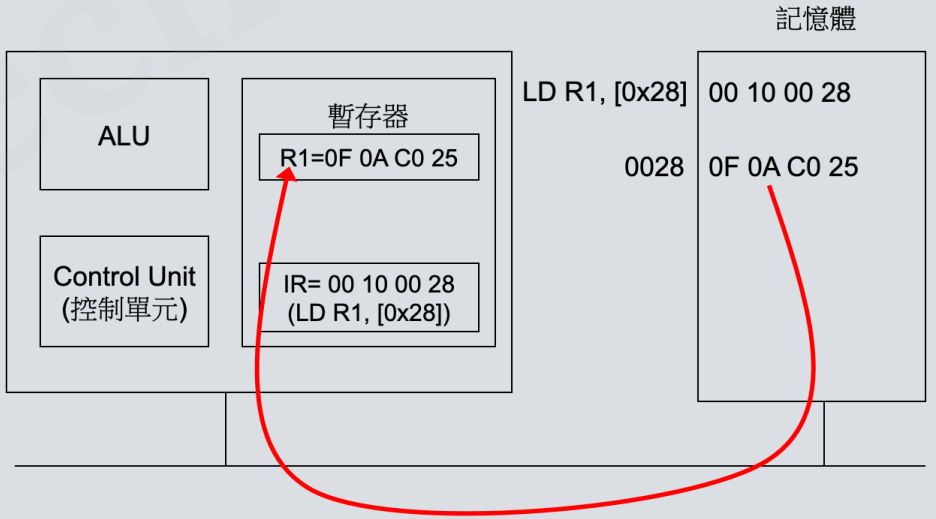
執行流程

1. CPU 取出指令。
2. 解碼指令以識別記憶體位址。
3. 將指定記憶體地址中的內容寫入到暫存器 R1。

IPSI Lab, Yuntech-CSIE

範例

`LD R1, [0x28]` 從記憶體位址 0x28 讀取一個字組並放入暫存器 R1



IPSI Lab, Yuntech-CSIE

ST 和 STB 指令（儲存指令）

- **ST (Store) 指令**
  - 將暫存器中的一個字組儲存到記憶體中的指定位址。
  - 格式：`ST Rs, [address]`
    - **Rs**：來源暫存器，儲存需要寫入記憶體的數值。
    - **address**：記憶體中的地址，**將數值寫入到該地址**。
- **STB (Store Byte) 指令**
  - 作用：將暫存器中的單一位元組寫入記憶體。
  - 使用場景：用於處理較小數值，如標記或旗標。

IPSI Lab, Yuntech-CSIE

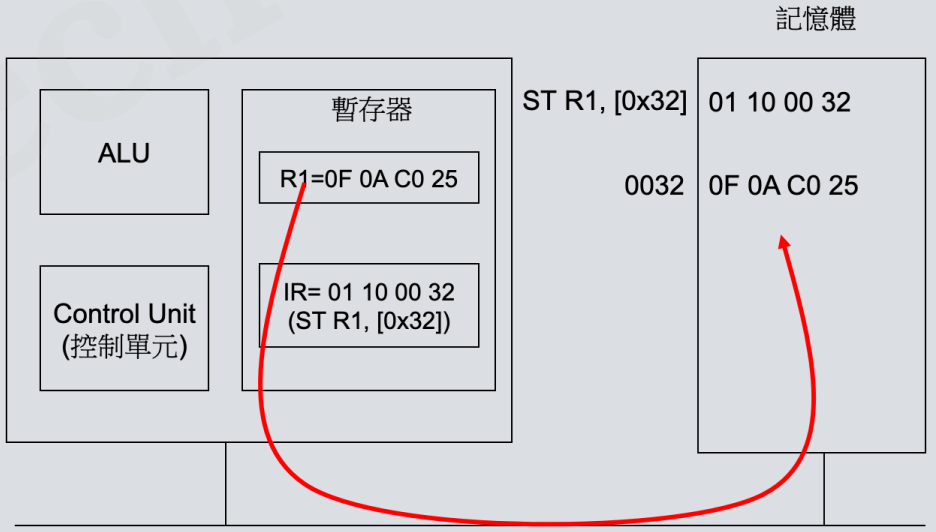
執行流程

1. CPU 取出指令。
2. 解碼指令以識別儲存的記憶體位址。
3. 將暫存器 R1 的內容寫入指定的記憶體位置。

IPSI Lab, Yuntech-CSIE

範例

`ST R1, [0x32]` 將暫存器 R1 中的字組資料寫入記憶體位址 0x32。



IPSI Lab, Yuntech-CSIE

暫存器與記憶體之間的交互操作

- **暫存器的作用**
  - 暫存器是CPU內部的高速存儲單元，用於臨時保存正在處理的數值。
  - **範例**：`LD R1, [R2 + 300]` 從 R2 所指定的記憶體位址加上偏移量300處，載入一個字組到 R1 中。

IPSI Lab, Yuntech-CSIE

暫存器間的交互操作，R 字母結尾指令

- 以字母 "R" 為後綴的指令 (如 LDR, STR) 表示這些指令使用暫存器進行位址計算。
- 例子：`LDR R1, [R2+R3]`。
  (可對比 `LD R1, [R2 + 300]`)。
- 當使用R 字母結尾指令時，CPU 可以進行靈活的記憶體操作，允許位址偏移和暫存器間數值的快速傳輸。

IPSI Lab, Yuntech-CSIE

移動指令（MOV Instruction）

IPSI Lab, Yuntech-CSIE

移動指令（MOV Instruction）

- **MOV 指令**用來將數值從一個暫存器移動到另一個暫存器。
- **格式**：`MOV Rd, Rs`
  - **Rd**：目標暫存器，負責儲存數值。
  - **Rs**：來源暫存器，提供數值。

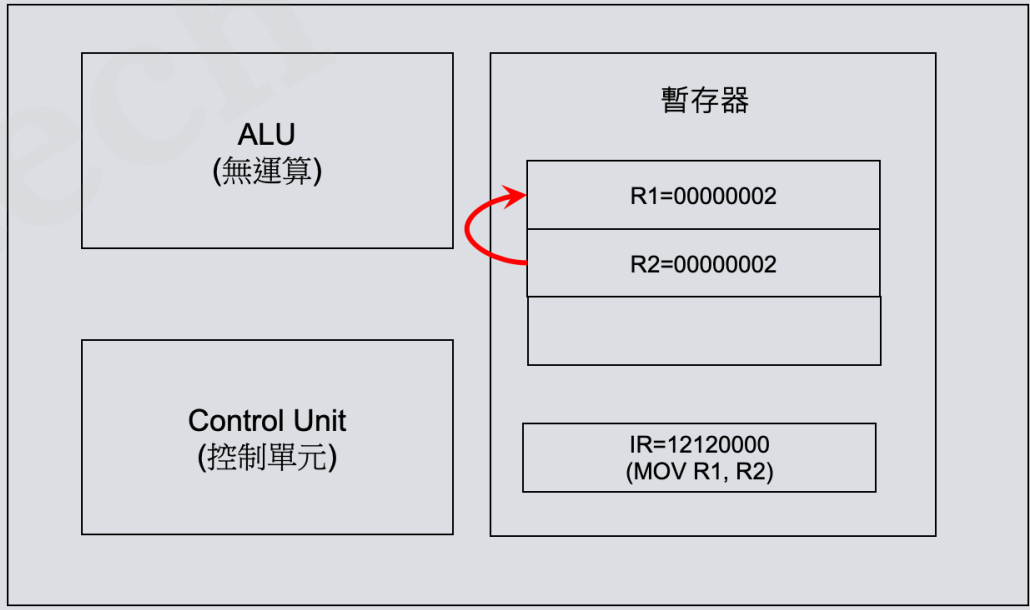
IPSI Lab, Yuntech-CSIE

執行流程

1. 從 來源暫存器 Rs 中讀取數值。
2. 將數值寫入 目標暫存器 Rd。

範例

- 指令：`MOV R1, R2`
  - 將 R2 中的數值 `0x00000002` 複製到 R1。
  - 結果：R1 = `0x00000002`，R2 = `0x00000002`（不變）。



算術指令（Arithmetic Instructions）

IPSI Lab, Yuntech-CSIE

算術指令（Arithmetic Instructions）

- **算術指令**用於進行數學運算，包括加法、減法等。
- **常見指令**：
  - **ADD**：加法運算，
    格式：`ADD Rd, Rs1, Rs2`（`Rd = Rs1 + Rs2`）
  - **SUB**：減法運算，
    格式：`SUB Rd, Rs1, Rs2`（`Rd = Rs1 - Rs2`）

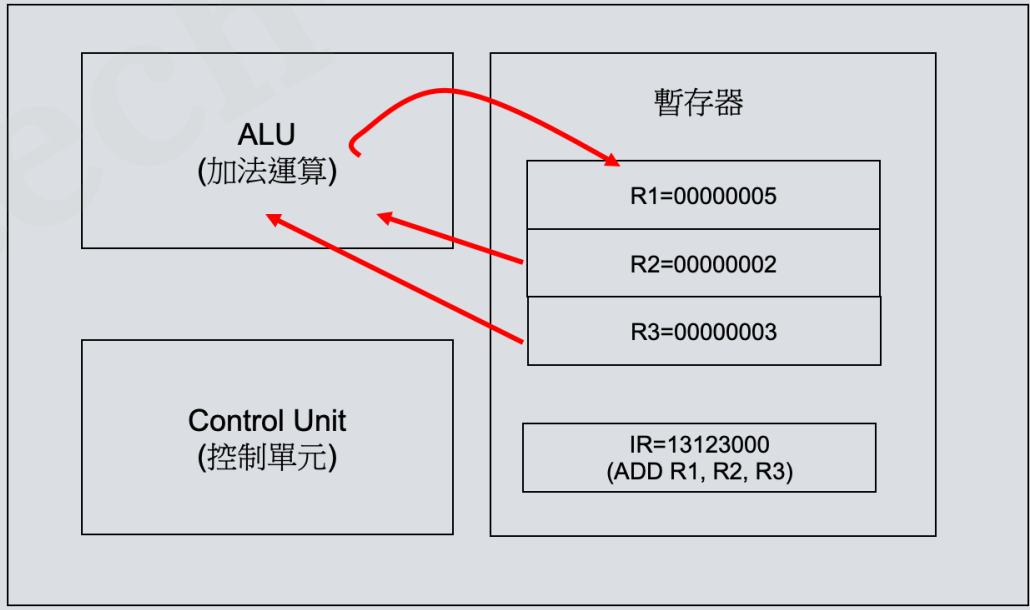
IPSI Lab, Yuntech-CSIE

加法執行流程

1. 從來源暫存器 Rs1 和 Rs2 中讀取數值。
2. ALU 執行加法運算，將結果存入目標暫存器 Rd。

範例：

- 指令：`ADD R1, R2, R3`
  - R2 = `0x00000002`，R3 = `0x00000003`
  - 結果：R1 = `0x00000005`

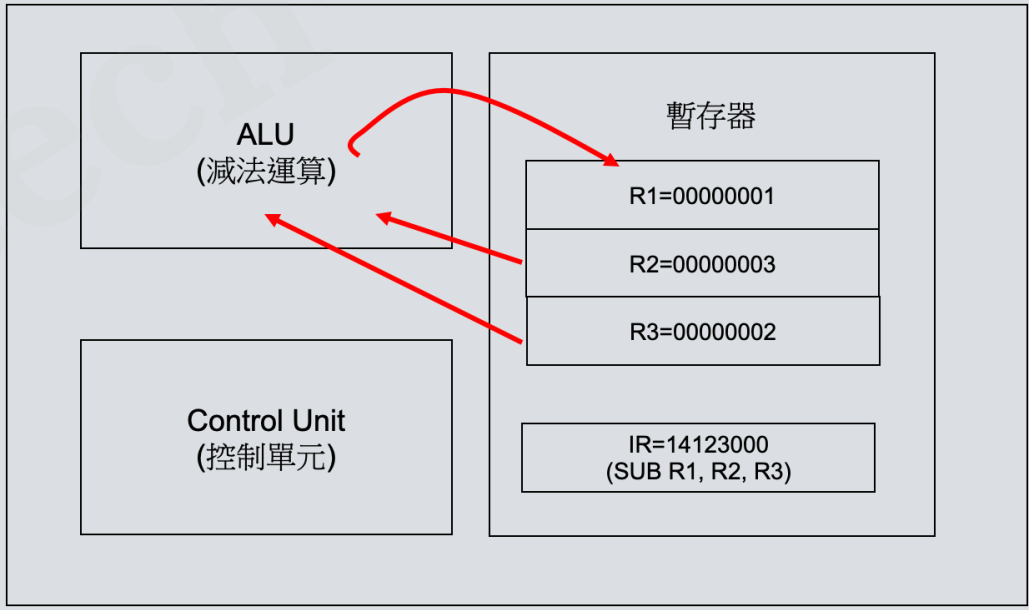


減法執行流程

- 流程同加法指令，不同的是在ALU 執行減法運算。

範例：

- 指令：`SUB R1, R2, R3`
  - R2 = `0x00000003` ，R3 = `0x00000001`
  - 結果：R1 = `0x00000002`



邏輯運算指令（Logical Instructions）

IPSI Lab, Yuntech-CSIE

邏輯運算指令（Logical Instructions）

- **邏輯指令**用於執行位元邏輯運算，如 AND、OR、XOR。
- **常見指令**：
  - **AND**：位元與運算，
    格式：`AND Rd, Rs1, Rs2`（`Rd = Rs1 & Rs2`）
  - **OR**：位元或運算，
    格式：`OR Rd, Rs1, Rs2`（`Rd = Rs1 | Rs2`）
  - **XOR**：位元異或運算，
    格式：`XOR Rd, Rs1, Rs2`（`Rd = Rs1 ^ Rs2`）

IPSI Lab, Yuntech-CSIE

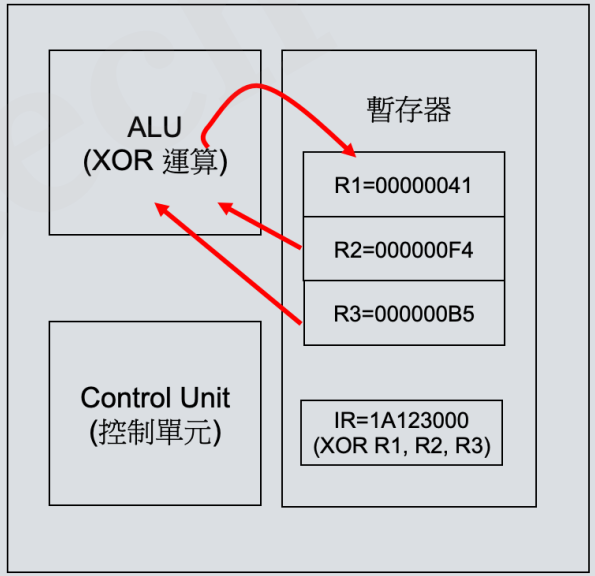
執行流程

1. 從來源暫存器 Rs1 和 Rs2 中讀取數值。
2. ALU 執行異或運算，將結果存入目標暫存器 Rd。

範例：

- 指令：`XOR R1, R2, R3`

1. R2 = `0x00000002` ，R3 = `0x00000003`
   - 結果：R1 = `0x00000001` （因為 2 XOR 3 = 1)
2. R2 = `0x000000F4` ，R3 = `0x000000B5` (右圖範例)
   - 結果：R1 = `00000041`



R1 (二進位) = * 0100 0001

R2 (二進位) = * 1111 0100

R3 (二進位) = * 1011 0101

＊ 符號，代表暫存器前面的24個位元被省略了

移位指令（Shift Instruction）

IPSI Lab, Yuntech-CSIE

移位指令（Shift Instruction）

- **移位指令**用於將暫存器中的數值進行位元移位操作。
- **常見指令**：
  - **SHL（左移）**：格式：`SHL Rd, Rs, #imm`，
    將 Rs 中的數值左移 imm 位，結果存入 Rd。
  - **SHR（右移）**：格式：`SHR Rd, Rs, #imm`，
    將 Rs 中的數值右移 imm 位，結果存入 Rd。
- 另有類似移位指令: **旋轉指令**，如: **ROL（左旋）**、**ROR（右旋）**。

#imm 代表的是「立即數」（Immediate），它是一個常數，用於指定左移或右移的位數。

執行流程

1. 從來源暫存器 Rs 中讀取數值。
2. ALU 將 Rs 中的數值左移 `#imm` 位，結果存入目標暫存器 Rd。

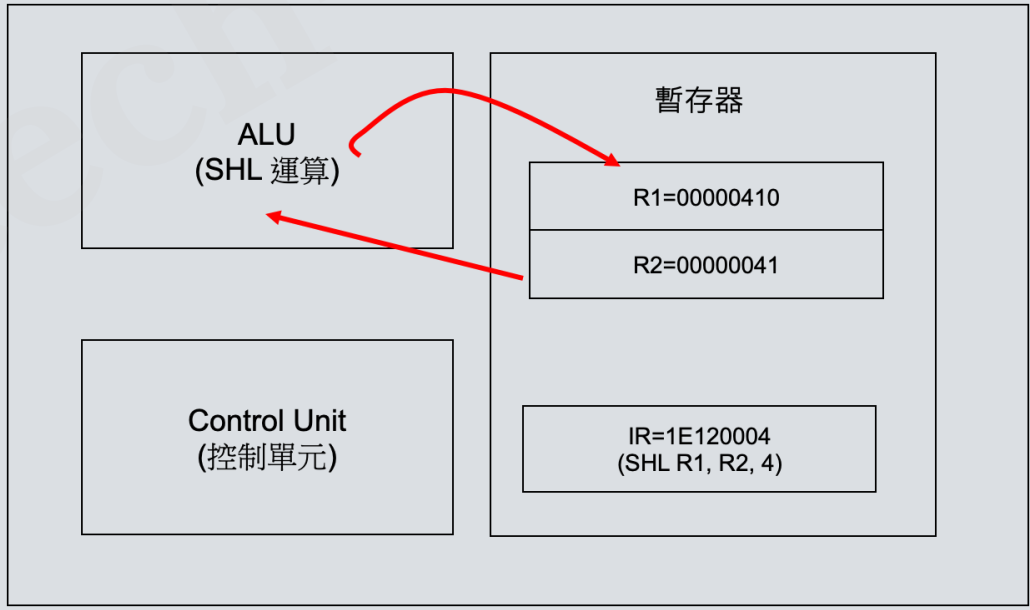
範例

- 指令：`SHL R1, R2, 4`

1. R2 = `0x00000002`

- `0x00000002` = `0000…0010` 左移 4 位 → `0000…0010 0000` = `0x00000020`
- 結果：R1 = `0x00000020`（因為 2 左移 4 位 = 32）

1. R2 = `0x00000041`（右圖範例）



- 暫存器 R2 的初始值是 `0x00000041`（也就是十六進位的 41，轉換成二進位是 `0100 0001`）。
- 左移 4 位元的操作會使數字擴大 2^4，也就是擴大 16 倍。
- 也就是將二進位 `0100 0001` 向左移 4 位元後變成：`0100 0001 0000`。
- 結果：R1 = `0100 0001 0000` 轉換成十六進位，就是 `0x000000410`

IPSI Lab, Yuntech-CSIE

比較指令 (CMP Instruction)

IPSI Lab, Yuntech-CSIE

比較指令 (CMP Instruction)

- **比較指令 (CMP) 是用來比較兩個暫存器中的值，並根據比較結果更新 狀態暫存器 (SW, Status Word)。**
- **雖然和算術指令看似類似，但 CMP 並不會將結果存入任何暫存器，它的作用是設置狀態旗標。**
- **格式：** `CMP Rd, Rs`
  - **Rd**：第一個操作數暫存器。
  - **Rs**：第二個操作數暫存器。

IPSI Lab, Yuntech-CSIE

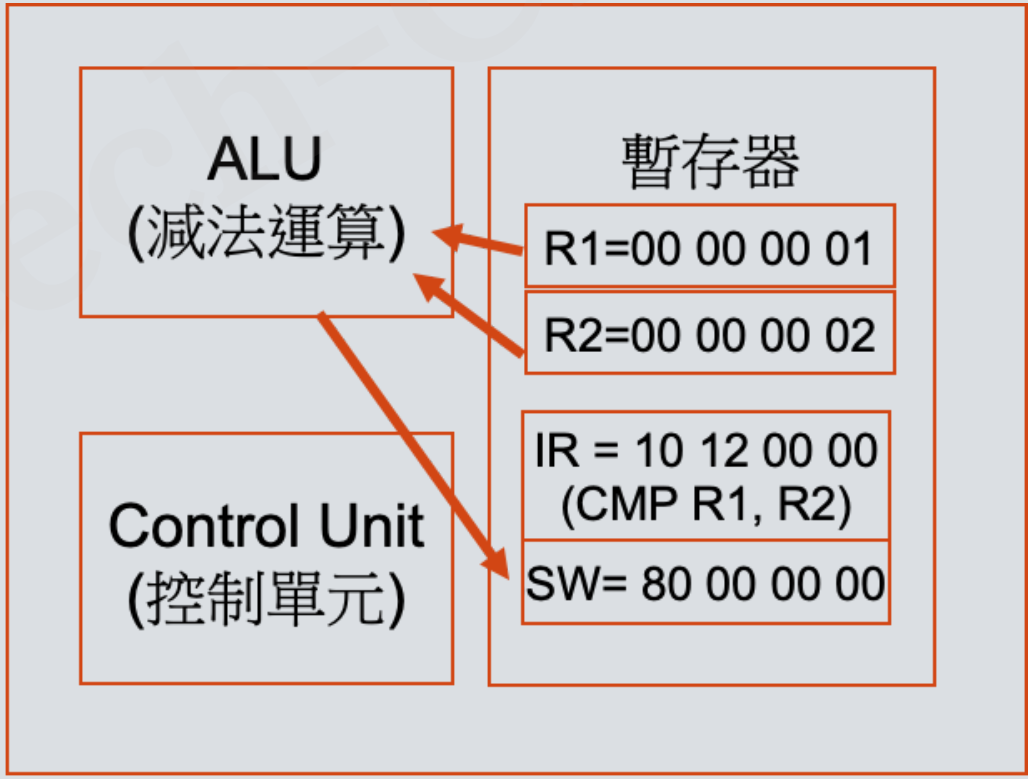
執行流程

1. 取出暫存器 **R1** 和 **R2** 中的值。
2. **ALU** 執行減法運算（R1 - R2），但不會將結果存入暫存器。
3. 根據減法運算的結果，更新 **狀態暫存器 SW** 中的旗標。

IPSI Lab, Yuntech-CSIE

比較指令範例：

- **指令**：`CMP R1, R2`
  - R1 = `0x00000001`，R2 = `0x00000002`
  - ALU 執行 `R1 – R2`，結果為負數（-1），因此：
    - **N = 1**，因為結果為負數。
    - **Z = 0**，因為結果不為零。
    - **C = 1**，因為沒有借位。
    - **V = 0**，沒有溢位發生。



比較指令後續之跳躍

- 條件旗標的 N、Z 旗標值, 可以用來代表比較結果, 當執行 `CMP Ra, Rb` 動作後, 可能會有下列三種情形。
  - 若 Ra > Rb, 則 N = 0, Z = 0。
  - 若 Ra < Rb, 則 N = 1, Z = 0。
  - 若 Ra = Rb, 則 N = 0, Z = 1。
- 如此, 用來進行**條件跳躍**的 JGT、JGE、JLT、JLE、JEQ、JNE 等指令, 就可以根據 N、Z 旗標決定是否進行跳躍。

IPSI Lab, Yuntech-CSIE

補充: 狀態暫存器 (SW) 的結構



- **狀態暫存器 (SW)** 用於儲存 CPU 執行比較指令或算術運算後的狀態標記。
- **SW 的四個條件旗標(Condition Code)**：
  - **N (Negative)**：當結果為負數時，N 被設置為 1。
  - **Z (Zero)**：當結果為 0 時，Z 被設置為 1。
  - **C (Carry)**：當沒有借位時，C 被設置為 1。
  - **V (Overflow)**：當發生溢位時，V 被設置為 1。

- **SW 中的中斷控制旗標**
  - **I (Interrupt) 旗標**：
    - 控制 CPU0 對所有中斷的反應。
    - 當 I 設為 0，CPU0 將停止響應所有中斷信號。
  - **T (Trap) 旗標**：
    - 用來控制軟體中斷（Software Interrupt, SWI）。
    - 當 T 設為 0 時，只禁止軟體中斷，不影響硬體中斷的運作。
- **SW 中的處理器模式欄位**
  - **M 欄位**：
    - M=0：代表「使用者模式」 (User Mode)
      - 使用者模式：限制對某些關鍵功能的修改，主要用於應用程式。
    - M=1：代表「特權模式」 (Super Mode)
      - 特權模式：允許對系統進行更高層級的操作，主要用於作業系統。

IPSI Lab, Yuntech-CSIE

補充：Interrupt & Trap （詳見作業系統課程）

Interrupt

- **定義**：
  中斷是來自 **處理器外部或內部的非同步事件**，打斷目前正在執行的程式，轉去執行「中斷服務程式（ISR, Interrupt Service Routine）」。一般是「非同步」於目前指令流程。
- **來源**：
  - **硬體中斷（外部）**：像是 I/O 裝置（鍵盤、計時器、網路卡）要求 CPU 服務。
  - **軟體中斷（內部）**：程式主動發出特定指令（例如 SWI, INT 指令）要求系統服務。

IPSI Lab, Yuntech-CSIE

Trap

- **定義**：
  Trap 是一種 **同步的軟體觸發事件**，通常由程式執行到某個特定指令或條件（例如 **SWI** 指令、系統呼叫、除零錯誤）時產生。
  它會強制 CPU 轉入「陷阱服務程式」。
- **來源**：
  - 軟體呼叫（系統呼叫、SWI 指令）。
  - 異常（Exception），像是非法指令、算術錯誤。

IPSI Lab, Yuntech-CSIE

補充：User Mode & Super Mode （詳見作業系統課程）

User Mode（使用者模式）

一種「受限」的處理器執行模式，專門給應用程式（User Program）使用。

- **限制**：
  - 不能存取敏感的硬體資源（例如中斷控制器、I/O 直接操作、記憶體管理單元設定）。
  - 不能修改處理器的控制暫存器（例如中斷允許旗標、模式切換旗標）。
  - 嘗試使用受限指令會產生陷阱（Trap），交由作業系統處理。
- **用途**：保護系統核心不被應用程式誤改，避免單一程式造成全系統崩潰。
- **舉例**：一般程式在 Windows/Linux/Android 下執行時，都是在 **User Mode**。

Super Mode（特權模式 / Kernel Mode）

一種「完全權限」的處理器模式，通常保留給作業系統核心（OS Kernel）。

- **能力**：
  - 可直接存取硬體（I/O 埠、裝置暫存器）。
  - 可修改處理器的控制暫存器（例如中斷旗標 I、Trap 旗標 T、模式欄位 M）。
  - 可執行所有 CPU 指令，包括受限指令。
- **用途**：提供作業系統管理硬體與系統資源的能力，例如排程、記憶體管理、檔案系統。
- **舉例**：Linux kernel、Windows NT 核心程式碼，就是在 **Super Mode** 下執行。

IPSI Lab, Yuntech-CSIE

跳躍指令 (Jump Instructions)

IPSI Lab, Yuntech-CSIE

跳躍指令 (Jump Instructions)

- **跳躍指令** 用於改變程式執行流程，通過改變程式計數器 **PC** 的值來實現跳躍到指定的指令位置。
- 跳躍指令分為：
  - **無條件跳躍指令**：直接跳躍到新的地址，不依賴任何條件。
  - **條件跳躍指令**：根據狀態暫存器 (SW) 中的旗標決定是否跳躍。

IPSI Lab, Yuntech-CSIE

無條件跳躍指令

- **無條件跳躍指令 (JMP)** 直接改變程式計數器 (PC) 的值，無條件跳躍到目標地址 (**註: 在此以相對跳躍方式**)。
- **格式**：`JMP address`
  - 最後跳躍的目標地址為 `PC+ address`

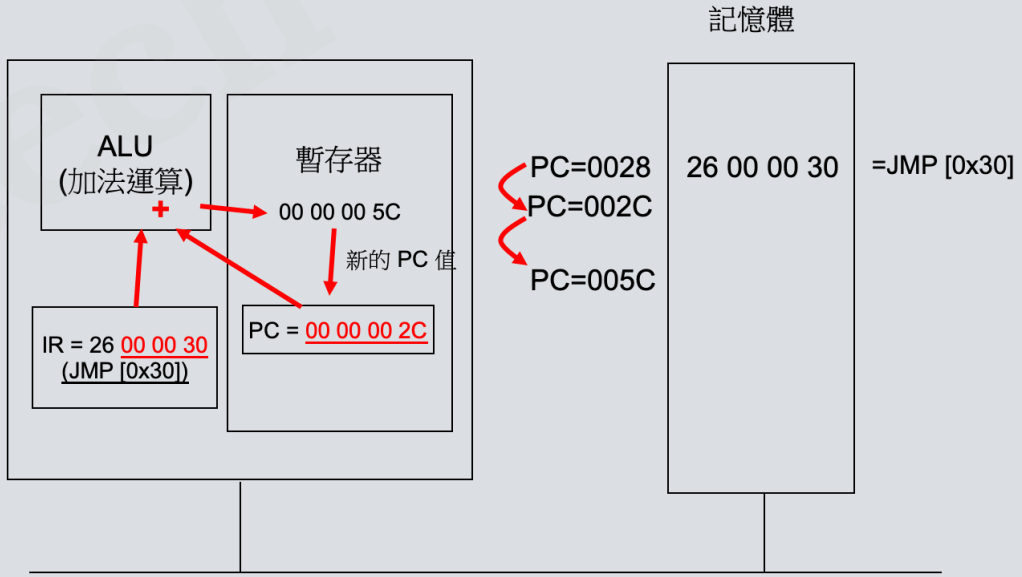
執行流程

1. 根據指令中的地址，將 **PC** 設定為目標地址。
2. 程式從目標地址繼續執行下一條指令。

IPSI Lab, Yuntech-CSIE

範例

- 指令：`JMP 0x30`
  - 指令提取後，PC從 `0x28` 變成 `0x2C` 。
  - `0x30` + `0x2C` = `0x5C`
  - 程式跳躍到 **`0x5C`** 地址，並從該處繼續執行。



條件跳躍指令

- **條件跳躍指令** 根據狀態暫存器 (SW) 中的旗標來決定是否跳躍 (**註: 在此以相對跳躍方式**)。
- 根據比較指令 **CMP** 的結果，設置不同的旗標來影響跳躍行為。

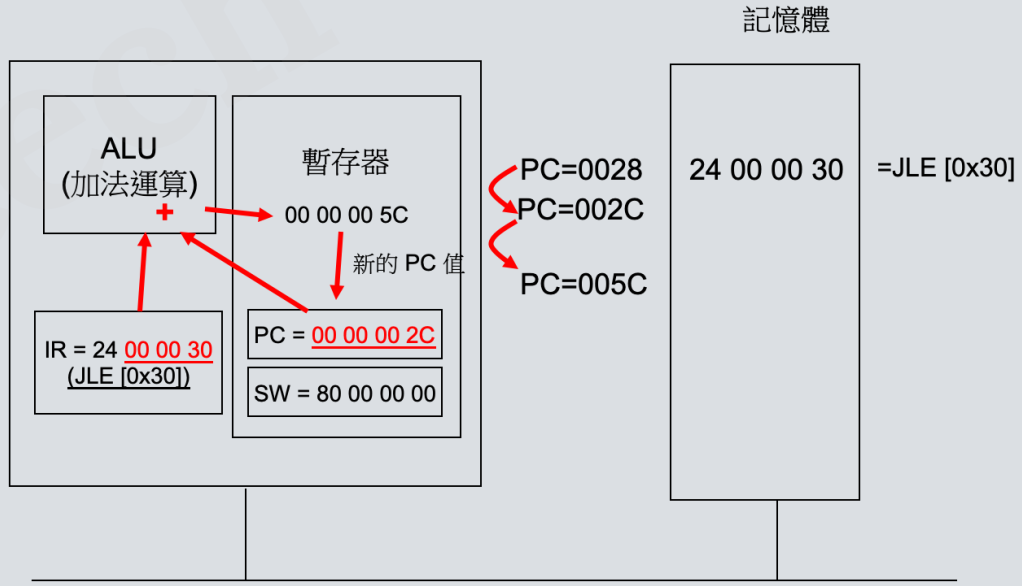
常見條件跳躍指令

- **JEQ (Jump if Equal)**
- **JNE (Jump if Not Equal)**
- **JGT (Jump if Greater Than)**
- **JLT (Jump if Less Than)**
- **JLE (Jump if Less or Equal)**

IPSI Lab, Yuntech-CSIE

範例

- 指令：`JLE 0x30`（右圖範例）
  - 查看SW = `0x 80 00 00 00`
  - 當比較結果小於或等於（N = 1 或 Z = 1）時，跳躍到相對位址 **0x30** 地址。
- 指令：`JEQ 0x30`
  - 指令提取與PC的運算如同上一個範例。
  - 當比較結果相等（Z = 1）時，跳躍到相對位址 **0x30** 地址。
  - 如果不相等，程式繼續執行下一條指令。
  - 查看SW = `0x 40 00 00 00` 中之 `NZCV` 為 4 = 0100，故跳躍到 **0x5C** 地址。



其他: 跳躍指令的設計方式

- **絕對跳躍 (Absolute Jump)**
  - 程式計數器 (PC) 直接設置為指令中給出的目標地址。
  - **範例**：`JMP 0x30`
    - 程式將直接跳躍到地址 **0x30** 處繼續執行。
- **相對跳躍 (Relative Jump)**
  - 跳躍地址是相對於當前 **PC** 的偏移量，而不是具體的絕對地址。
  - **範例**：假設 **PC = 0x2C**，指令 `JMP 0x30` 意味著跳躍到當前 **PC + 0x30**，即新 **PC = 0x5C**。

IPSI Lab, Yuntech-CSIE